CSE 331: MICROPROCESSOR INTERFACING & Embedded Systems

Lecture 2: RISC vs CISC

CISC

- CISC: complex instruction set computer
- But original CPUs very simple
- Poorly suited to evolving high level languages
- Extended with new instructions
  - e.g. function calls, non-linear data structures, array bound checking, string manipulation
- Implemented in *microcode (which are RISC like)*
  - sub-machine code for configuring CPU sub-components
  - 1 machine code == > 1 microcode
- e.g. Intel X86 architecture

CISC continued

- CISC introduced integrated combinations of features
  - e.g. multiple address modes, conditional execution
  - required additional circuitry/power
  - multiple memory cycles per instruction
  - over elaborate/engineered
    - many feature combinations not often used in practise
- Could lead to loss of performance
- Better to use combinations of simple instructions

Towards CISC

- Wired logic $\rightarrow$ microcode control
  - Temptingly easy extensibility
- Performance tuning
  - HW implementation of some high-level functions
- Marketing
  - Add successful instructions of competitors
  - “New feature” hype
  - Compatibility: only extensions are possible

CISC Problems

- Performance tuning unsuccessful
  - Rarely used high-level instructions
  - Sometimes slower than equivalent sequence
- High complexity
  - Pipelining bottlenecks $\rightarrow$ lower clock rates
  - Interrupt handling can complicate even more
- Marketing
  - Prolonged design time and frequent microcode errors hurt competitiveness

RISC

- RISC: reduced instruction set computer
- return to simpler design
- general purpose instructions with small number of common features
    - less circuitry/power
    - execute in single cycle
- code size grew
- performance improved
- e.g. ARM, MIPS

RISC Features

- Low complexity
  - Generally results in overall speedup
  - Less error-prone implementation by hardwired logic or simple microcodes
- VLSI implementation advantages
  - Less transistors
  - Extra space: more registers, cache
- Marketing
  - Reduced design time, less errors, and more options increase competitiveness

RISC Compiler Issues

- The compilers themselves
  - Computationally more complex
  - More portable
- The compiler writer
  - Less instructions $\rightarrow$ probably easier job
  - Simpler instructions $\rightarrow$ probably less bugs
  - Can reuse optimization techniques

RISC vs. CISC *misconceptions*

- Arguments favoring RISC: simple design, short design time, speed, price…
- Study of RISC should include hardware/software tradeoffs, factors influencing computer performance and industry-side evaluation.

RISC vs. CISC *misconceptions*

- Incorrect implication from the two acronyms: RISC and CISC.
  - They are not bifurcations between which designers have to choose
- Carelessly leaving out the 'participation' of Operating System

RISC vs. CISC *misconceptions*

- Reduced design time?
  - academic <-> industrial
- Performance claims of RISC proponent do not decouple design features like MRSs (Model Specific Register).
  - MRSs can have a remarkable effect on program execution

Conclusion – RISC vs. CISC?

- **The Performance Equation**
  The following equation is commonly used for expressing a computer's performance ability:

$$\frac{\text{time}}{\text{program}} = \frac{\text{time}}{\text{cycle}} \text{ x } \frac{\text{cycles}}{\text{instruction}} \text{ x } \frac{\text{instructions}}{\text{program}}$$

- The CISC approach attempts to minimize the number of instructions per program, sacrificing the number of cycles per instruction. RISC does the opposite, reducing the cycles per instruction at the cost of the number of instructions per program.

Conclusion – RISC vs. CISC?

- CISC
  - *Effectively realizes one* particular High Level Language Computer System *in HW* - recurring **HW development costs** when change needed
- RISC
  - *Allows effective realization of any* High Level Language Computer System *in SW* - recurring **SW development costs** when change needed

Conclusion – Optimum?

- Hybrid solutions
  - RISC core & CISC interface
  - Still has *specific performance tuning*
- Optimal ISA
  - Between RISC & CISC
  - Few, carefully chosen, useful complex instructions
  - Still has *complexity handling problems*

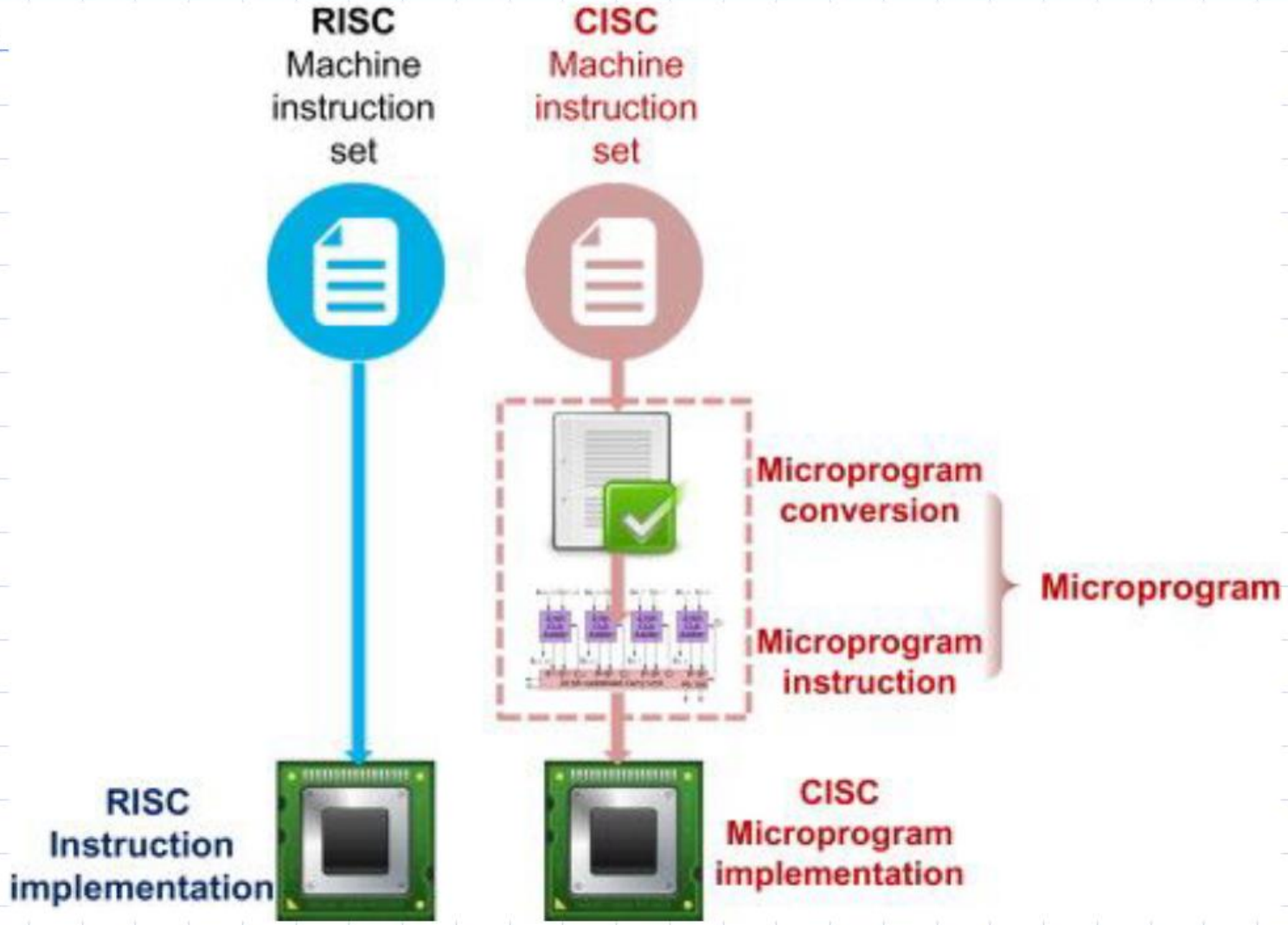
CISC vs RISC

RISC	CISC
It is a Reduced Instruction Set Computer.	It is a Complex Instruction Set Computer.
It focuses on Software.	It focuses on Hardware.
It uses only a Hardwired control unit.	It uses both hardwired and microprogrammed control units.
Transistors are used for more registers.	Transistors are used for storing complex instructions.
Code size is large.	Code size is small.
The uses of the pipeline are simple in RISC.	Uses of the pipeline are difficult in CISC.
An instruction is executed in a single clock cycle.	Instruction may take more than one clock cycle.
An instruction can fit in one word.	Instructions are larger than the size of one word.
The execution time of RISC is very short.	The execution time of CISC is longer.
The program written for RISC architecture needs to take more space in memory.	The Program written for CISC architecture tends to take less space in memory.

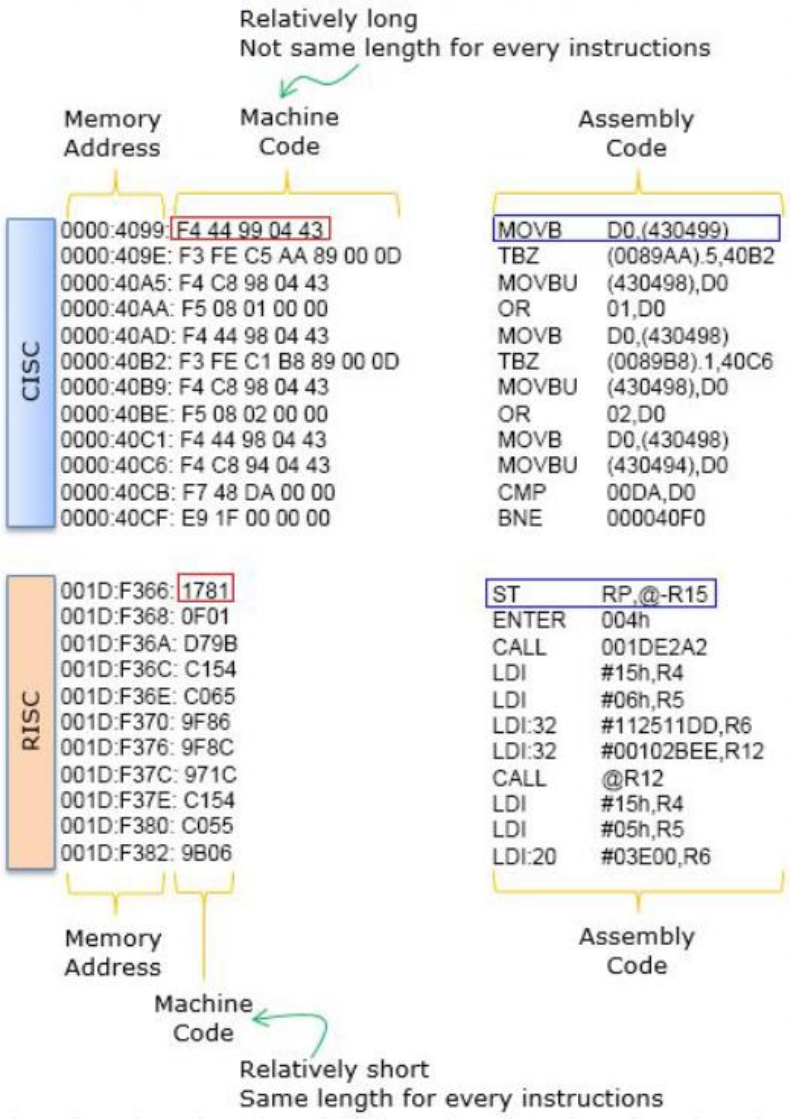
RISC Vs CISC

Parameters	RISC	CISC
Full Form	❖ Reduced Instruction Set Computer	❖ Complex Instruction Set Computer
Instruction Size	❖ Fixed Size	❖ Variable Size
Instruction Fetch	❖ Same for all instruction	❖ Vary with respect to instructions
Instruction Set	❖ Small & Simple	❖ Large and Complex
Addressing Modes	❖ Less Modes as most instructions are based on registers	❖ More Modes as Complex instructions are available with different verities.
Numbers of Registers	❖ Many	❖ Few
Complier Design	❖ Simple	❖ Complex
Program Size	❖ Long {Weak code density}	❖ Small {Better Code density}
Numbers of Operand	❖ Fixed {Mainly in Registers}	❖ Variable {Can be in Registers & Memory}
Control Unit	❖ Hardwire controlled	❖ Micro Program Controlled
Execution Speed	❖ Faster	❖ Slower
Pipelining	❖ More Effective	❖ Less Effective {It has more bubbles due more memory based instructions}

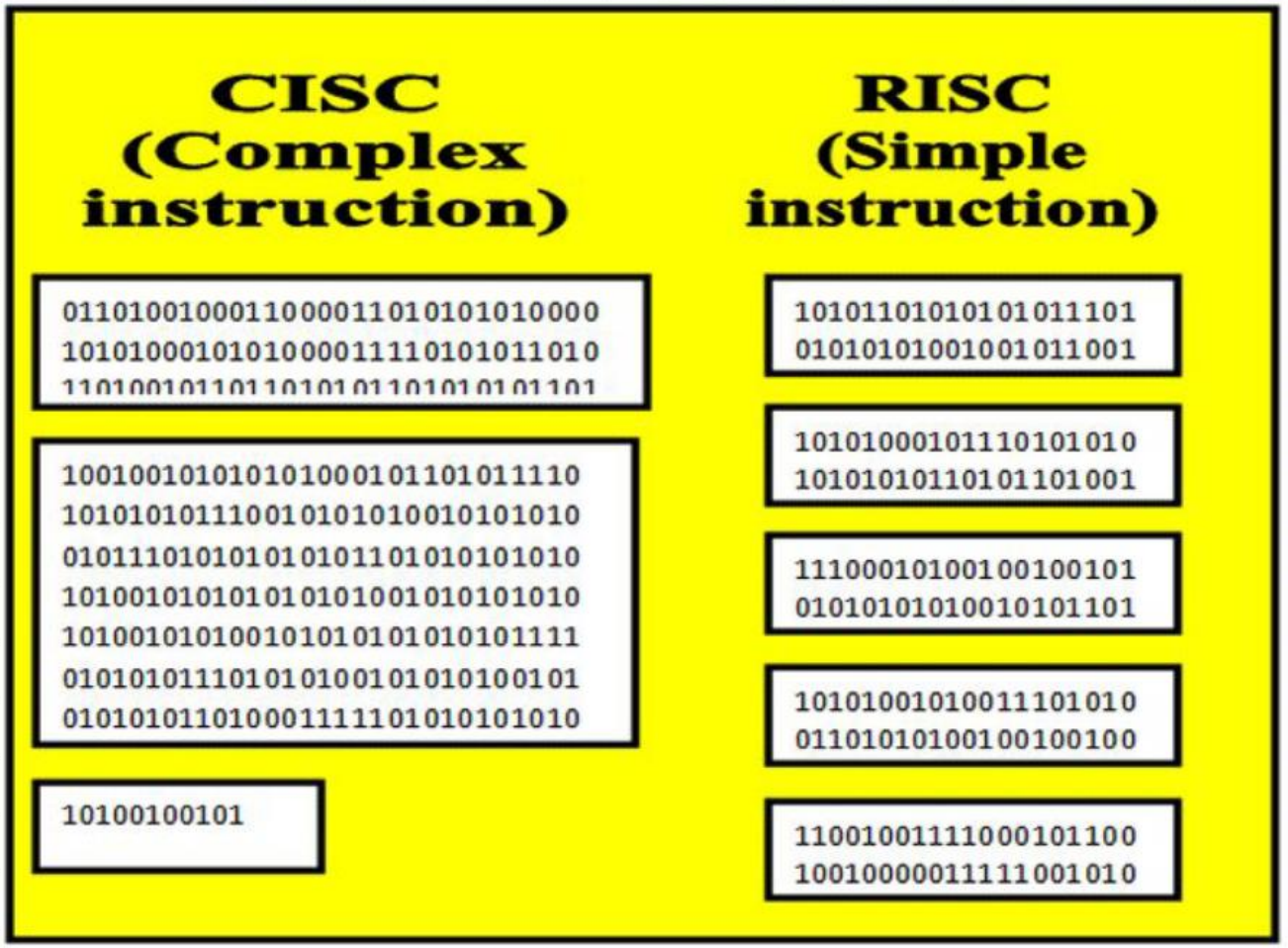
CISC vs RISC


RISC
Machine
instruction
set
CISC
Machine
instruction
set
Microprogram
conversion
Microprogram
instruction
Microprogram
RISC
Instruction
implementation
CISC
Microprogram
implementation

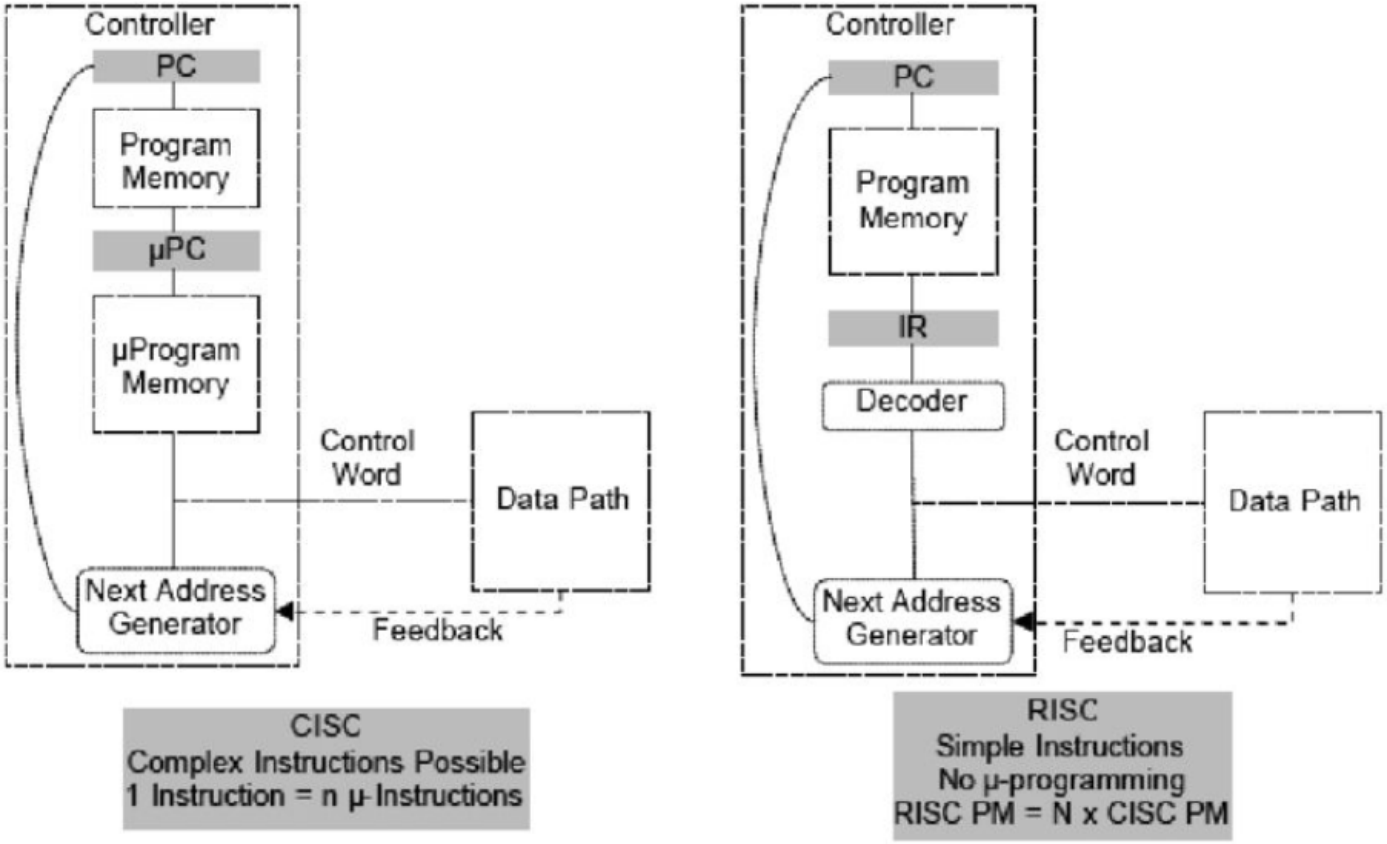
CISC vs RISC


Relatively long
Not same length for every instructions
Memory Address
Machine Code
Assembly Code
CISC
0000:4099: F4 44 99 04 43
0000:409E: F3 FE C5 AA 89 00 0D
0000:40A5: F4 C8 98 04 43
0000:40AA: F5 08 01 00 00
0000:40AD: F4 44 98 04 43
0000:40B2: F3 FE C1 B8 89 00 0D
0000:40B9: F4 C8 98 04 43
0000:40BE: F5 08 02 00 00
0000:40C1: F4 44 98 04 43
0000:40C6: F4 C8 94 04 43
0000:40CB: F7 48 DA 00 00
0000:40CF: E9 1F 00 00 00
MOVB D0,(430499)
TBZ (0089AA).5,40B2
MOVBU (430498),D0
OR 01,D0
MOVB D0,(430498)
TBZ (0089B8).1,40C6
MOVBU (430498),D0
OR 02,D0
MOVB D0,(430498)
MOVBU (430494),D0
CMP 00DA,D0
BNE 000040F0
RISC
001D:F366: 1781
001D:F368: 0F01
001D:F36A: D79B
001D:F36C: C154
001D:F36E: C065
001D:F370: 9F86
001D:F376: 9F8C
001D:F37C: 971C
001D:F37E: C154
001D:F380: C055
001D:F382: 9B06
ST RP,@-R15
ENTER 004h
CALL 001DE2A2
LDI #15h,R4
LDI #06h,R5
LDI:32 #112511DD,R6
LDI:32 #00102BEE,R12
CALL @R12
LDI #15h,R4
LDI #05h,R5
LDI:20 #03E00,R6
Memory Address
Assembly Code
Machine Code
Relatively short
Same length for every instructions

CISC vs RISC


CISC
(Complex
instruction)
01101001000110000110101010100000
10101000101010000111101011010
10100100101010100010110101110
10101010111001010100101010
01011101010101010110101010
10100101010101010100101010
10100101010010101010101111
01010101110101010010101010
01010101101000111110101010
10100100101
RISC
(Simple
instruction)
10101101010101011101
01010101001001011001
10101000101110101010
10101010110101101001
11100010100100100101
01010101010010101101
10101001010011101010
01101010100100100100
11001001111000101100
10010000011111001010

CISC vs RISC


Controller
PC
Program Memory
μPC
μProgram Memory
Control Word
Data Path
Next Address Generator
Feedback
CISC
Complex Instructions Possible
1 Instruction = n μ-Instructions
Controller
PC
Program Memory
IR
Decoder
Control Word
Data Path
Next Address Generator
Feedback
RISC
Simple Instructions
No μ-programming
RISC PM = N x CISC PM